Lesson 1: Overview of Computer Graphics

Lesson 1: Overview of Computer Graphics 1
1.1 Definition of terms and conventions 2
1.1.1 Graphics 2
1.1.2 Computer 2
1.1.3 OpenGL 2
1.2 History of Computer Graphics 2
1.2.1 Sub topic 2
1.3 Relationship between mathematics and Computer Graphics 2
1.4 Raster Displays 3
1.4.1. Explanation on how in a raster system pixels are displayed 3
1.5 Basic Line Drawing 4
Lesson 1 Discussion Session 4

1.1 Definition of terms and conventions

1.1.1 Graphics

A **graphic** is an image or visual representation of an object. Therefore, computer **graphics** are simply images displayed on a computer screen.

Computer graphics is a discipline concerned with all aspects of producing pictures or images using a computer.

Computer graphics is the discipline of generating images with the aid of computers.

1.1.2 Computer

A **computer** is a machine or device that performs processes, calculations and operations based on instructions provided by a software or hardware program. It is designed to execute applications and provides a variety of solutions by combining integrated hardware and software components.

1.1.3 OpenGL

OpenGL is a Graphics software system for 3D computer graphics programming.
It is a worldwide standard for 3D computer graphics programming.
OpenGL library - a 3D graphics programming library that enables you to create high performance graphics applications.
OpenGL is easy to learn, and it possesses most of the characteristics of other popular graphics systems.

1.2 History of Computer Graphics

1.2.1 Sub topic

Computer Graphics (CG) was first created as a visualization tool for scientists and engineers in government and corporate research centers such as Bell Labs and Boeing in the 1950s. Later the tools would be developed at Universities in the 60s and 70s at places such as Ohio State University, MIT, University of Utah, Cornell, North Carolina and the New York Institute of Technology.
The early breakthroughs that took place in academic centers continued at research centers such as the famous Xerox PARC in the 1970s. These efforts broke first into broadcast video graphics and then major motion pictures in the late 70s and early 1980s. Computer graphic research continues today around the world, now joined by the research and development departments of entertainment and production companies. Companies such as George Lucasąs Industrial Light and Magic are constantly redefining the cutting edge of computer graphic technology in order to present the world with a new synthetic digital reality.

1.3 Relationship between mathematics and Computer Graphics

Virtually all computing devices in use today are digital; data is represented in a discrete form using patterns of binary digits (bits) that can encode numbers within finite ranges and with limited precision. By contrast, the images we perceive in our environment are analogue. They are formed by complex interactions between light and physical objects, resulting in continuous variations in light wavelength and intensity.
All Computer Graphics demands a working knowledge of linear algebra (matrix manipulation, linear systems, etc.) as the system can be represented in digital format (using sequence of 0s and 1s). Much of

Computer Graphics involves discussion of points in 2D or 3D. Usually we write such points as Cartesian Coordinates e.g. p = [x, y]T or q = [x, y, z]T. Point coordinates are therefore vector quantities.
By manipulating the points, we can define the shape of an object, or move it around in space.
In 3D too, we can model a shape using points. Points might define the locations (perhaps the corners) of surfaces in space.
A common application of these techniques is to apply a matrix transformation to a 2D image, e.g. to rotate, skew, or scale a "source" (input) image to create a modified "target" (output) image. This process is known as **digital image warping** and was the motivating example given in Figure 3.1.



Figure 3.1.

1.4 Raster Displays

The screen is represented by a 2D array of locations called **pixels**.



Zooming in on an image made up of pixels

Each pixel corresponds to a location, or small area, in the image. Collectively, the pixels are stored in a part of memory called the **frame buffer**.
The conversion of geometric entities to pixel colors and locations in the frame buffer is known as **rasterization** or **scan conversion.** Simply **rasterization** or **scan conversion** is the process of converting basic, low level objects into their corresponding pixel map representations. This is often an approximation to the object, since the frame buffer is a discrete grid.

1.4.1. Explanation on how in a raster system pixels are displayed

In a raster system, the graphics system takes pixels from the frame buffer and displays them as points on the surface of the display in one of two fundamental ways.
In a **non-interlaced** system, the pixels are displayed row by row, or scan line by scan line, at the refresh rate.
In an **interlaced** display, odd rows and even rows are refreshed alternately. Interlaced displays are used in commercial television. In an interlaced display operating at 60 Hz, the screen is redrawn in its entirety only 30 times per second, although the visual system is tricked into thinking the refresh rate is 60 Hz rather than 30 Hz. Viewers located near the screen, however, can tell the difference between

the interlaced and non-interlaced displays. Non-interlaced displays are becoming more widespread, even though these displays process pixels at twice the rate of the interlaced display

1.5 Basic Line Drawing

Set the color of pixels to approximate the appearance of a line from (x_0, y_0) to (x_1, y_1).
It should be
• "straight" and pass through the end points.
• independent of point order.
• uniformly bright, independent of slope.
The explicit equation for a line is $y = mx + b$.

Note:
Given two points (x_0, y_0) and (x_1, y_1) that lie on a line, we can solve for m and b for the line.
Consider $y_0 = mx_0 + b$ and $y_1 = mx_1 + b$.
Subtract y0 from y_1 to solve for $m = (y_1 - y_0)/(x_1 - x_0)$
and
$b = y_0 - mx_0$.
Substituting in the value for b, this equation can be written as $y = m(x - x_0) + y_0$.

Consider this simple line drawing algorithm:

```
int x
float m, y
m = (y1 - y0) / (x1 - x0)
for (x = x0; x <= x1; ++x) {
  y = m * (x - x0) + y0
  setpixel(x, round(y), linecolor)
}
```

Lesson 1 Discussion Session

1. Discuss frame buffer as used in Computer graphics (discussion should be at least half a page).
2. Explain how the Cathode Ray Tube works (include a CRT diagram in the explanation)